Exercise 15

Ex. No.: 15

Detect SQL Injection Vulnerabilities using Various SQL Injection Detection Tools

Aim

Use sqlmap to automatically detect (non-destructively) whether input points on a web application are

vulnerable to SQL injection, produce verifiable evidence, and collect information needed to

remediate the issue — all within an isolated lab.

Prerequisites & tools

Target: a local/test web app you control (e.g., DVWA, Juice Shop, or your dev site).

Download SQLMAP

Unzip go that folder

Cd C:\Users\balus\Downloads\sqlmap

Python sqlmap.py -h

python sqlmap.py -u "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" --

cookie="PHPSESSID=g833nc5k1lcj0nnggvumnfj1o4; security=low" --proxy="http://localhost:8080"

-dbs

security=low; PHPSESSID=g833nc5k1lcj0nnggvumnfj1o4

-D dvwa –tables

sqlmap installed (latest stable) on your attacker/test machine.

Intercepting proxy (Burp/ZAP) — optional but highly recommended for identifying injectable

parameters first.

Browser configured to proxy through ZAP/Burp (or use ZAP's browser) so you can capture traffic.

Start with non-destructive detection settings — do not run extraction options (--dump, --dbs, --tables,

etc.) unless you explicitly intend to and have authorization.

Log and timestamp your tests and retain evidence for remediation.

High-level procedure (recommended workflow)

Map the app with proxy

Browse the app while ZAP/Burp is running and capture all requests. Identify pages and parameters

that take user input (GET/POST parameters, cookies, headers).

Prioritize inputs that reach the server and are echoed or used in SQL-backed pages (ID parameters,

search fields, login forms).

Passive analysis & manual sanity checks

With proxy, send simple benign tricky inputs (e.g., ' or " or very long strings) to suspected parameters

and observe responses and server error messages. If the app returns DB errors or changes response

structure, mark it for automated testing.

Note exact request format (method, URL, parameter name) — sqlmap needs this info.

Run sqlmap in detection-only, non-destructive mode

Use sqlmap to detect injectable parameters without extracting data. Make sure you do not include

extraction flags (--dump, --dbs, --tables, --columns, --os-shell, etc.).

Example (lab / detection-only):

```
sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli/?id=1" \
 --batch --level=1 --risk=1 --threads=1 --random-agent --identify-waf
```

Explanation of choices:

-u target URL (change to the exact request/parameter you captured).

--batch run non-interactively (uses default answers).

--level=1 --risk=1 keeps tests minimal and less intrusive (safer for a lab).

--threads=1 single thread to reduce load.

--random-agent rotates User-Agent to avoid trivial blocks.

--identify-waf (optional) makes sqlmap attempt to detect basic WAFs — useful for interpretation.

If the input is POST or requires special headers/cookies, feed sqlmap the request file captured from

your proxy:

Save the full raw HTTP request from Burp/ZAP to a file req.txt.

Run:

sqlmap -r req.txt --batch --level=1 --risk=1 --threads=1

-r lets sqlmap use the exact request (method, headers, body, cookies). Good for complex forms or

authenticated requests.

Interpret sqlmap output (detection-only)

sqlmap will report which parameters it believes are injectable and the (non-destructive) tests that

succeeded. Typical output lines:

"Parameter 'id' appears to be injectable (boolean-based blind)"

"it is safe to resume with --batch" or detected payload types/techniques.

Important: Treat automated findings as potential — manually verify high/critical items (replay the

test request with a trivial payload and observe response behavior) before reporting.

Manual verification & evidence collection

Reproduce the finding manually via your proxy (send benign probe that triggers the same response

pattern). Capture request/response pairs and screenshots for the report.

Do not extract databases or sensitive data unless explicitly required and authorized.

Triage & report

For each vulnerable parameter record: endpoint, HTTP method, parameter name, injection type (as

sqlmap reports), commands used for detection (logged), and safe reproduction steps.

Provide remediation recommendations (parameterized queries, prepared statements, input validation,

least-privilege DB accounts, error suppression).

Example: Using a saved request file (safe approach)

In Burp, right-click a captured request → Copy as file → save as req.txt.

Run (detection-only):

sqlmap -r req.txt --batch --level=1 --risk=1 --threads=1 --random-agent

Read output: sqlmap lists tests it ran and whether the parameter is injectable. No data is dumped.

Never add extract flags: --dump, --dbs, --tables, --columns, --os-shell.

Use low --level/--risk values for detection (1). Higher values increase payload complexity and

intrusiveness.

Use --batch so sqlmap uses safe defaults, and --timeout/--threads to limit load.

Keep --threads low and avoid high-concurrency scans on production.

Interpreting common sqlmap findings

parameter appears to be injectable (type) — indicates sqlmap found evidence using its tests (boolean-

based, time-based, error-based, UNION-based, etc.). Use manual verification to confirm context and

impact.

back-end DBMS: ... — sqlmap may fingerprint the DB engine. This helps remediation but also

indicates you should tighten DB account privileges.

HTTP error / WAF detected — may indicate either a protection blocking tests or the app returning

errors; analyze with proxy.

Remediation checklist (what to do when a vulnerability is confirmed)

Fix code: replace string-concatenated SQL with parameterized/prepared statements (PDO, ADO.NET

SqlCommand with parameters, parameter binding in your stack).

Validate and normalize input server-side (whitelist acceptable values).

Limit DB account privileges (no dbo/sysadmin for web app accounts).

Suppress detailed DB error messages in production; log internally.

Implement Web Application Firewall/IPS as an additional layer (but do not rely on it alone).

Add monitoring/alerts for anomalous query patterns or repeated probes.

Sqlmap output snippets showing detection (not full dumps).

Manual reproduction steps (safe probes that confirm the behavior).

Remediation recommendations and suggested priority (High/Medium/Low).

Result:

If sqlmap reports nothing but you suspect vulnerability, increase --level gradually (2→3) in the lab

only — each increase runs more payload types/tests.

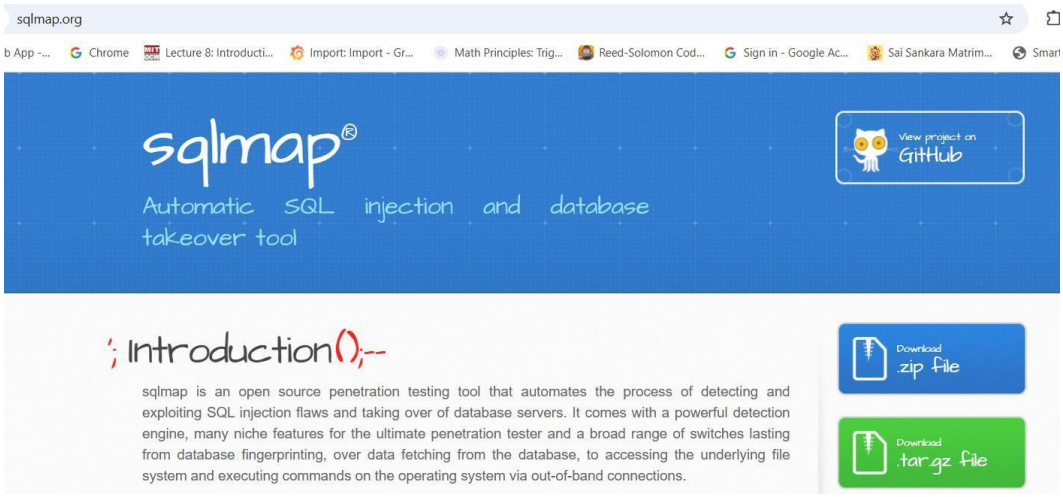
If the app uses anti-CSRF tokens or requires a login, use -r req.txt with a captured authenticated

request or configure sqlmap's --cookie/--data/--headers options.

If a WAF is interfering, capture WAF responses in proxy and use that info to tune the scan, or test in

a more isolated environment.

Screenshots




```
power@SHIBU-HOME MINGW64 ~/Desktop/sqlmap
$ python sqlmap.py -u "http://dvwa.test/vulnerabilities/sqli/?id=1&Submit
=Submit" --cookie="PHPSESSID=g076cnolube4qpmin7vp174or5; security=low" --
proxy="http://localhost:8080"
```

```
[12:38:33] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL >= 5.6
[12:38:33] [INFO] fetching database names
available databases [13]:
[*] binod
[*] cmsysv2
[*] cmsysv2_new
[*] cmsysv4
[*] dhundo
[*] dvwa
[*] information_schema
[*] mocker
[*] mysql
[*] payments_rcoe
[*] performance_schema
[*] rcoe_moodle
[*] sys
```